TP 8 : Un problème de sac à dos

Ce TP est à réaliser en `OCaml`.

Préliminaires

On considère le problème suivant :

`Sac à dos`	Instance :	Une liste d'objets caractérisés par leur poids et leur valeur (p_i, v_i)
		Un entier P, la capacité maximale du sac à dos P
		Un entier V, la valeur minimale que l'on souhaite transporter
	Question :	Est-ce qu'il est possible de choisir des objets dont le poids total est inférieur à P mais dont la valeur totale est supérieure à V ?

FIGURE 1 – Une bonne journée qui commence

L'objectif de ce TP est de proposer des algorithmes pour essayer de le résoudre. Pour cela, on s'intéresse dans un premier temps au problème d'optimisation associé : maximiser la valeur sans excéder P.

Définitions

On définit une instance du problème par le type `instance` suivant :

```
type instance =
{
  nombre_objets : int;
  tableau_objets : (int * int) array;
  p_max : int;
  v_min : int;
}
```

Pour chaque objet, on associe un couple p_i, v_i.
Étant donnée une instance I, on note $v_{OPT}(I)$ la valeur maximale d'une sélection d'objets valide.

Approche naïve

Dans un premier temps, on se propose d'implémenter un algorithme naïf basé sur une relation de récurrence. On remarque que les solutions utilisant une instance sont partitionnées entre celles qui contiennent le i-ième élément et celles qui ne le contiennent pas. Pour caractériser une solution, on renvoie un tableau de booléens indiquant si l'objet i est dans la solution.

Question 1 : Écrire une fonction récursive `eval_aux : instance -> int -> int` qui calcule la valeur maximale d'un sous-ensemble ne dépassant pas `p_max` d'une instance restreint à ses i premiers objets.

Question 2 : En déduire une fonction `evaluation_naive : instance -> int` qui calcule la valeur maximale que l'on peut stocker dans le sac caractérisé par l'instance.

Programmation dynamique

On cherche à améliorer la complexité de l'algorithme précédent en utilisant la programmation dynamique. On note $V(i,p)$ la plus grande valeur d'un sous-ensemble des i premiers objets de poids au plus p.

Pour cela, on introduit un tableau à deux dimensions de type `int array array` en `OCaml`. On remarque que :

$$\forall i, V(i+1, p+p_i) = \max\{V(i, p+p_i), V(i,p)+v_i\}$$

Question 3 : En étudiant les conditions d'initialisation, écrire une fonction `creer_tableau : instance -> int array array` qui crée et initialise le tableau nécessaire à la programmation dynamique.

Question 4 : Écrire une fonction `remplir : int array array -> instance -> unit` qui remplit le tableau à l'aide de la relation de récurrence.

Question 5 : Écrire une fonction `evaluation_dynamique_valeur : instance -> int` qui calcule la valeur maximale d'objets que l'on peut stocker dans le sac.

Question 6 : Écrire une fonction `trouver_solution : instance -> bool array` qui étant donnée une instance, renvoie une solution maximisant la somme totale.

Question 7 : On note $P(i,v)$ le plus petit poids d'un sous-ensemble des i premiers objets de valeur totale v. Écrire une fonction `evaluation_dynamique_poids : instance -> int` qui exploite la programmation dynamique sur ces valeurs.

Algorithme glouton

Pour s'assurer d'avoir un algorithme polynomial en la taille de l'entrée, on utilise un algorithme glouton. Celui-ci ajoute les objets au sac à dos suivant leur ratio $\left(\frac{v_i}{p_i}\right)$. On fait le choix d'ajouter les objets les plus rentables en termes de rapport valeur/poids.

Question 8 : Écrire une fonction `trier_objet : (int*int)array -> unit` qui, étant donné un tableau d'objets, le trie selon ce rapport.

Question 9 : Écrire une fonction `glouton : instance -> int` qui renvoie la valeur du contenu du sac pour une instance donnée.

Question 10 : On veut montrer que les valeurs renvoyées par cet algorithme peuvent être éloignées de la valeur optimale. Soit v_G la valeur renvoyée par l'algorithme glouton. Pour tout entier K, construire une instance I telle que $v_{OPT}(I)/v_G(I) > K$.

```
let exemple_instance =
{
  nombre_objets = 5;
  tableau_objets = [| (2, 3); (3, 4); (4, 5); (5, 8); (9, 10) |];
  p_max = 10;
  v_min = 15;
}
```